7 DE OCTUBRE DE 2026

PRESUPUESTO Y GASTOS

GRUPO 8

Matías Mendoza (MM)
Jerissel Sanchez (JS)
Jorge Cabezas (JC)

UCOM250_81 - ORGANIZACIÓN Y ARQUITECTURA DE COMPUTADORES
2026

Contenido

Descripción 3

Análisis 3

Implementación 4

Evidencias de ejecución 5

Conclusiones 6

Bibliografía 6

Descripción

Escenario. El escenario asignado al Grupo 8 corresponde al control de un presupuesto disponible y los gastos registrados por una persona. El programa parte de un presupuesto de 100 y gastos de 135. La solución debe trabajar con estos valores almacenados en memoria, cargarlos en registros, efectuar el procesamiento requerido y determinar si existe una diferencia presupuestaria.

Resultado esperado. La versión final debe comparar el presupuesto y los gastos. Como 100 y 135 son diferentes, el programa debe identificar que existe una diferencia presupuestaria y mostrar por pantalla el texto “Existe diferencia presupuestaria: ” seguido del resultado de gastos menos presupuesto. Para los valores dados, el resultado esperado es 35.

Análisis

Los datos del escenario se almacenan inicialmente en memoria mediante variables de tipo .word. También se reservan dos posiciones para guardar el resultado aritmético y el resultado de la comparación.

Tabla 1. Datos del programa

Datos	Valor inicial	Propósito	Resultado esperado
dato1	100	Presupuesto disponible	Se carga en $t0
dato2	135	Gastos registrados	Se carga en $t1
resultado1	0	Reserva para la diferencia	35 en la versión final
resultado2	0	Reserva para la comparación	1, porque 100 != 135

Tabla 2. Operaciones requeridas

Datos / Resultados	Propósito	Operación requerida	Instrucción MIPS
dato1	Cargar presupuesto desde memoria	Carga	lw
dato2	Cargar gastos desde memoria	Carga	lw
resultado1	Calcular gastos - presupuesto	Resta	sub
resultado2	Comprobar si los valores son diferentes	Comparación	sne
Control de flujo	Finalizar si no existe diferencia	Salto condicional	beq
Resultados	Guardar valores calculados	Almacenamiento	sw
Salida	Mostrar mensaje y diferencia	Impresión	syscall

Implementación

La solución se desarrolló en dos etapas. La versión base permitió comprobar la carga de datos, la resta, la comparación y el almacenamiento de resultados. La versión final ajustó la operación al requerimiento de “gastos - presupuesto” e incorporó el control de flujo y la presentación del resultado por pantalla.

Versión base

El archivo version_base/Grupo 8 miniproyecto - Presupuestos y Gastos- MM JS JC.s carga el presupuesto y los gastos mediante lw. En esta primera versión se realiza la operación presupuesto - gastos, obteniendo -35. Después, sne determina si los datos son diferentes y sw guarda ambos resultados en memoria. Esta etapa sirvió como punto de partida para validar el recorrido de los datos desde memoria hacia los registros y de regreso a memoria.

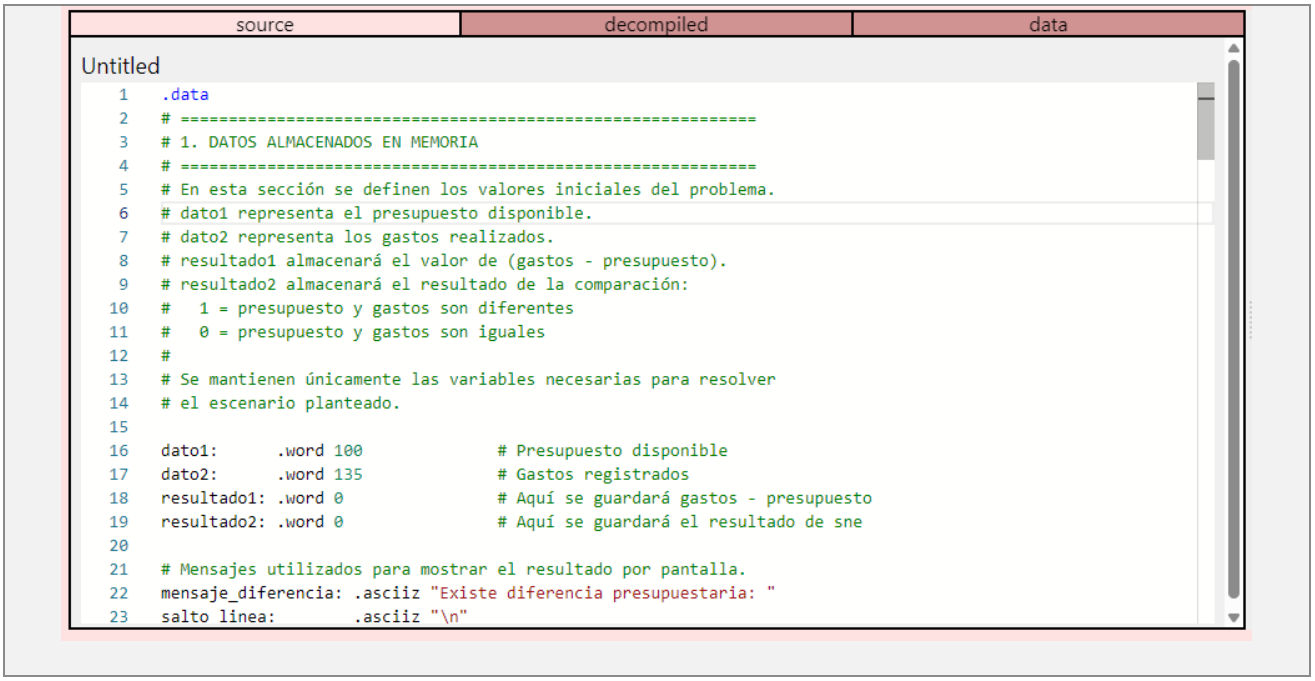
Archivo: version_base/Grupo 8 miniproyecto - Presupuestos y Gastos- MM JS JC.s

Versión final

El archivo version_final/Grupo_8_Avance_2_Presupuesto_Gastos_MM,JS,JC.s conserva la carga de los datos mediante lw, pero modifica la resta para calcular gastos - presupuesto, de acuerdo con el resultado requerido por pantalla. Así, 135 - 100 produce 35. La instrucción sne coloca 1 en $t3 porque los valores son diferentes. Posteriormente, sw almacena 35 en resultado1 y 1 en resultado2. La instrucción beq controla el flujo: si la comparación indicara igualdad, el programa terminaría sin mostrar el mensaje. Como sí existe diferencia, se ejecutan las llamadas syscall que imprimen el texto y el valor 35.

Archivo: version_final/Grupo_8_Avance_2_Presupuesto_Gastos_MM,JS,JC.s

Figura 1. Captura del código final en MIPSY Web



Untitled

```
26  .text
27  .globl main
28
29  main:
30
31      # ========================================================
32      # 2. CARGA DE DATOS DESDE MEMORIA MEDIANTE lw
33      # ========================================================
34      # lw significa "Load Word".
35      # La instrucción carga una palabra de 32 bits desde memoria
36      # hacia un registro.
37      #
38      # $t0 recibirá el presupuesto.
39      # $t1 recibirá los gastos.
40
41      lw $t0, dato1                   # $t0 = 100  (presupuesto)
42      lw $t1, dato2                   # $t1 = 135  (gastos)
43
44
45      # ========================================================
46      # 3. OPERACIÓN ARITMÉTICA
47      # ========================================================
48      # El escenario solicita mostrar el resultado de:
```

source | decompiled

Untitled

```
45      # ========================================================
46      # 3. OPERACIÓN ARITMÉTICA
47      # ========================================================
48      # El escenario solicita mostrar el resultado de:
49      #
50      #       gastos - presupuesto
51      #
52      # Como gastos = 135 y presupuesto = 100:
53      #
54      #       135 - 100 = 35
55      #
56      # El resultado se guarda temporalmente en $t2.
57
58      sub $t2, $t1, $t0             # $t2 = 135 - 100 = 35
59
60
61      # ========================================================
62      # 4. COMPARACIÓN ENTRE PRESUPUESTO Y GASTOS
63      # ========================================================
64      # sne significa "Set if Not Equal".
65      #
66      # La instrucción compara los valores de $t0 y $t1:
67      #
```

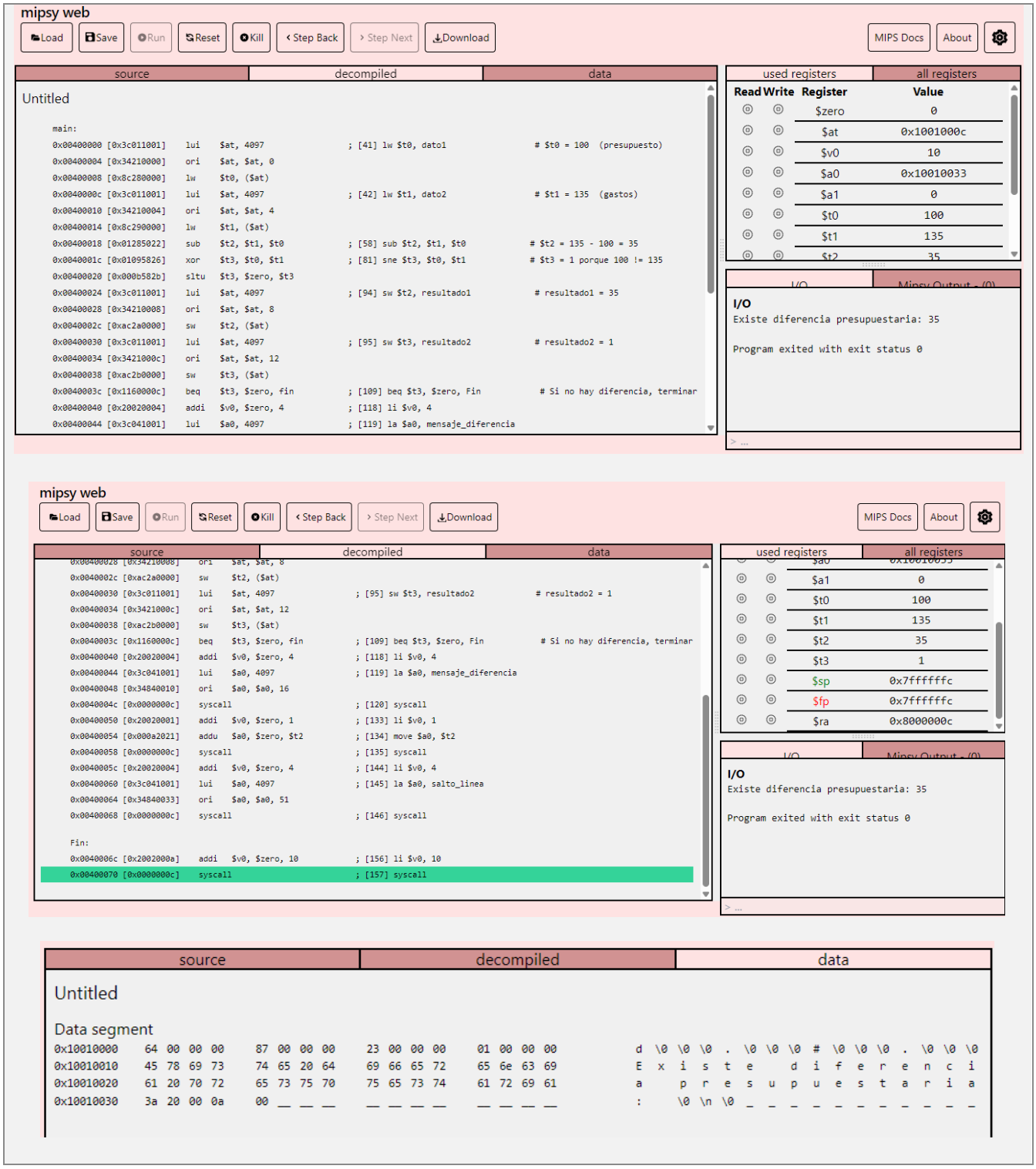
Untitled

```
60
61      # ========================================================
62      # 4. COMPARACIÓN ENTRE PRESUPUESTO Y GASTOS
63      # ========================================================
64      # sne significa "Set if Not Equal".
65      #
66      # La instrucción compara los valores de $t0 y $t1:
67      #
68      #       $t0 != $t1
69      #
70      # Si son diferentes, $t3 recibe 1.
71      # Si son iguales, $t3 recibe 0.
72      #
73      # En este caso:
74      #
75      #       100 != 135
76      #
77      # Por lo tanto:
78      #
79      #       $t3 = 1
80
81      sne $t3, $t0, $t1             # $t3 = 1 porque 100 != 135
```

Evidencias de ejecución

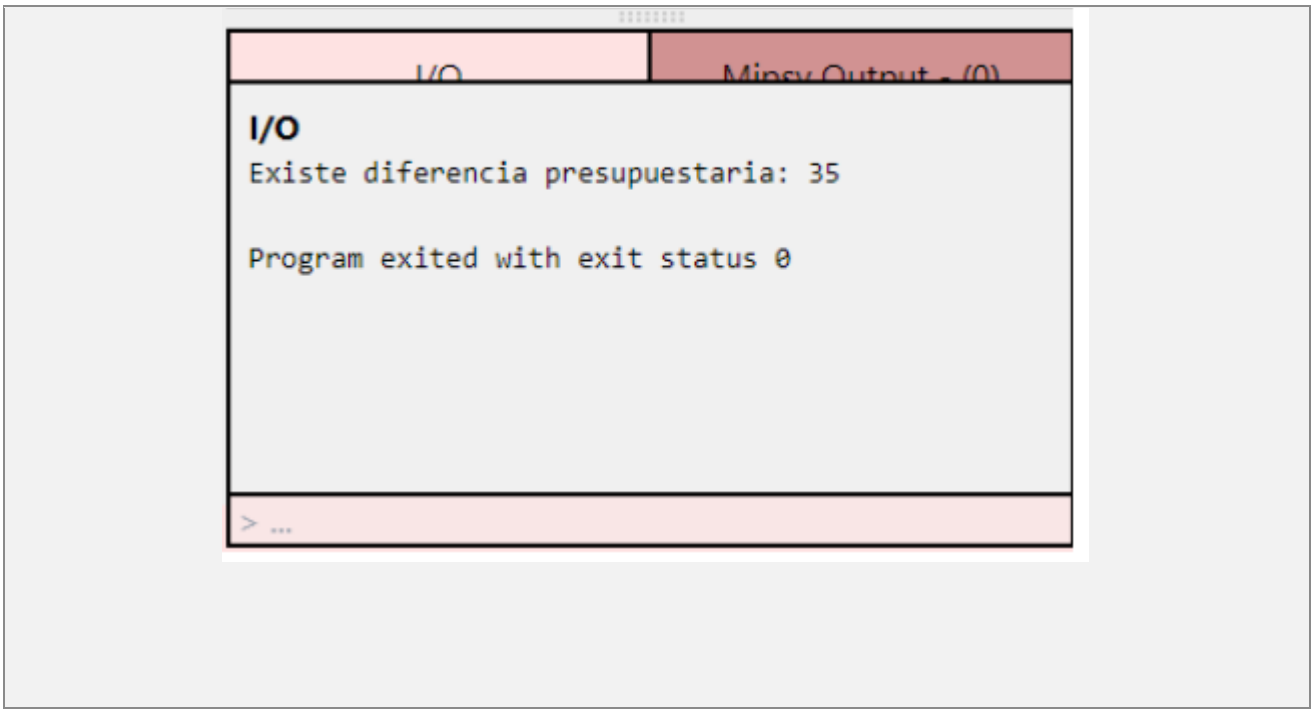
En esta sección se deben insertar las capturas tomadas por el grupo. Los nombres sugeridos coinciden con la estructura del repositorio de GitHub y con las referencias utilizadas en el README.md.

Figura 2. Registros después de la ejecución



En la captura de registros se espera comprobar los valores principales de la versión final: $t0 = 100 para el presupuesto, $t1 = 135 para los gastos, $t2 = 35 para la diferencia y $t3 = 1 como resultado de la comparación de desigualdad.

Figura 3. Resultado mostrado por pantalla



La salida esperada del programa es: “Existe diferencia presupuestaria: 35”. Esta evidencia permite comprobar que el control de flujo habilita la impresión únicamente cuando los valores son distintos y que el resultado numérico corresponde a gastos menos presupuesto.

Conclusiones

El desarrollo del proyecto permitió comprender de manera práctica cómo un problema sencillo puede dividirse en pasos y representarse mediante instrucciones de bajo nivel. La actividad ayudó a relacionar el planteamiento del escenario con la secuencia real que sigue el programa durante su ejecución, lo que facilitó entender mejor el comportamiento de un programa en lenguaje ensamblador.

Una de las principales dificultades fue seguir el recorrido de los datos y comprobar qué valor debía existir en cada etapa de la ejecución. Trabajar por bloques y verificar cada parte antes de continuar permitió resolver las dudas y detectar con mayor facilidad los errores. La comparación entre la versión base y la versión final también permitió observar cómo una solución puede evolucionar a partir de nuevos requerimientos.

La implementación práctica con el simulador hizo que los conceptos vistos en clase fueran más claros, ya que fue posible observar paso a paso el efecto de las instrucciones y relacionarlas con el resultado final. Si se realizara nuevamente la actividad, sería conveniente organizar desde el inicio las pruebas, capturas y documentación, de manera que cada avance quede respaldado y sea más sencillo construir la entrega final.

Bibliografía

University of New South Wales. (n.d.). MIPS instruction set. https://cgi.cse.unsw.edu.au/~cs1521/current/resources/mips-guide.html

Patterson, D. A., & Hennessy, J. L. (2021). Computer organization and design: The hardware/software interface (6th ed.). Morgan Kaufmann.

MARS. (n.d.). MIPS Assembler and Runtime Simulator. http://courses.missouristate.edu/kenvollmar/mars/